
Web Research Agent Using ReAct Pattern with LangChain and Gemini

1. Reasoning Step using LLM (Gemini via LangChain)

In this project, we used Google's Gemini model through LangChain to implement the ReAct (Reasoning + Acting) pattern.

Reasoning Phase (Planning)

- The agent begins by asking the Gemini LLM to generate 5–6 diverse research questions for a given user topic.
- The model analyzes the topic and breaks it into subcomponents for exploration.
- This step is purely LLM-driven and represents the "reasoning" part of the ReAct loop.

Prompt Example:

Generate 5–6 well-structured research questions about the topic: "Impacts of Generative AI on Education".

2. Code Structure and Execution Flow

Setup

- Required libraries: `langchain`, `google-generativeai`, `google-search-results`

- API Keys for Gemini (`GOOGLE_API_KEY`) and SerpAPI (`SERPAPI_API_KEY`) are loaded.

Tools

- A custom LangChain tool `SerpAPISearchTool` is defined using Pydantic v2 standards.
- It performs web search using SerpAPI for given queries (research questions).

LLM

- `ChatGoogleGenerativeAI` from LangChain is used to connect to Gemini and generate reasoning-based questions.

Agent Initialization

- The agent is initialized using `initialize_agent()` with type `ZERO_SHOT_REACT_DESCRIPTION`.
- This enables the LLM to choose and use tools (like search) on its own.

Report Compilation

- For each question, the agent runs a search and fetches summarized information.
- These are compiled into a Markdown report with an intro, sections for each question, and a conclusion.

Flow Summary

1. User inputs topic
2. LLM generates research questions
3. For each question:
 - Agent runs web search using tool
 - Gathers key results
4. A structured Markdown report is created and saved

This process demonstrates the ReAct pattern by combining planning (reasoning via Gemini) and acting (searching via SerpAPI).
